
Kubernetes Workloads (Detailed Overview)

1 Pod

The **smallest and most basic deployable unit** in Kubernetes.

A Pod wraps one or more **closely related containers**, sharing the same **network namespace** and **storage volumes**.

◆ Key Features:

- Containers in a Pod share IP, port space, and volumes.
- Designed to run **single instances** of applications.
- Managed directly or by higher-level controllers (e.g., Deployments).

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

This creates a Pod running a single NGINX container.

2 ReplicaSet

Ensures that a **specified number of replicas (Pods)** are running at any given time.

♦ **Key Features:**

- Automatically replaces failed or deleted Pods.
- Rarely used directly—usually managed via a **Deployment**.

 Example:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
```

This ensures 3 nginx Pods are always running.

Deployment

A high-level controller for **stateless applications**, offering **declarative updates**, rollbacks, and versioning.

♦ **Key Features:**

- Supports rolling updates and rollbacks.
- Creates and manages ReplicaSets under the hood.
- Ideal for microservices and frontends.

Example:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
```

Deploys 3 replicas of the nginx container and manages updates/rollbacks.

4 **StatefulSet**

Used for **stateful applications** requiring **persistent storage** and **stable identities**.

◆ **Key Features:**

- Provides stable hostnames (e.g., **pod-0**, **pod-1**, ...).

- Maintains **ordered** deployment, scaling, and deletion.
- Often paired with **PersistentVolumeClaims**.

 Example:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "web"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: [ "ReadWriteOnce" ]
        resources:
          requests:
            storage: 1Gi
```

Creates stateful NGINX pods with persistent storage.

5 DaemonSet

Ensures a **copy of a Pod runs on every node** (or a subset of nodes).

◆ Key Features:

- Useful for running node-local services (e.g., log collectors, monitoring agents).
- Automatically adds/removes Pods as nodes are added/removed.

Example:

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  selector:
    matchLabels:
      name: fluentd
  template:
    metadata:
      labels:
        name: fluentd
    spec:
      containers:
        - name: fluentd
          image: fluentd:latest
```

This deploys the Fluentd log collector on every node in the cluster.

6 Job

Creates Pods that **run to completion** and then exit. Used for **batch processing** or one-time tasks.

◆ Key Features:

- Ensures a task runs once (or to a specified completion count).
- Automatically retries on failure (configurable).

Example:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      containers:
        - name: hello
          image: busybox
          command: ["echo", "Hello Kubernetes"]
      restartPolicy: Never
```

Runs a single container to echo a message, then exits.

CronJob

Schedules **Jobs to run periodically** based on **Cron syntax**.

◆ Key Features:

- Time-based job scheduling (like Linux cron).
- Creates a Job at the defined time.

Example:

```
apiVersion: batch/v1
kind: CronJob
```

```
metadata:
  name: hello-cron
spec:
  schedule: "*/5 * * * *" # Every 5 minutes
jobTemplate:
  spec:
    template:
      spec:
        containers:
          - name: hello
            image: busybox
            command: ["date"]
        restartPolicy: OnFailure
```

Prints the current date every 5 minutes.

Summary Table

 Workload Type	Description	When to Use
① Pod	Smallest unit of deployment	For simple, standalone containers
② ReplicaSet	Ensures fixed number of Pods	When you want self-healing but no updates (rarely used alone)
③ Deployment	Manages stateless apps with updates/rollbacks	For scalable, stateless web apps and APIs
④ StatefulSet	Stateful apps with identity & storage	For databases, Zookeeper, etc.
⑤ DaemonSet	Runs on all (or some) nodes	For monitoring, logging, or network agents
⑥ Job	Run once and complete	Batch tasks, data processing
⑦ CronJob	Scheduled jobs using Cron	Reports, backups, periodic jobs
